

GenAI & Agents

Complete Career Roadmap — Post ML/DL/NLP

Sequential · Job-market aligned · No fluff

■ 18–24 weeks

■ 6 Phases

■ Interview-ready after
Phase 3

■ Mid-level after Phase 5

Phase 1 · LLM Foundations & Prompt Engineering

Weeks 1–3

WHY THIS PHASE

Everything downstream depends on knowing how these models actually work — context windows, tokenization, temperature, top-p. Skipping this means you'll debug blind later.

WHAT TO LEARN

- Transformer decoder-only architecture (conceptual depth, not math)
- Tokenization: BPE, SentencePiece, tiktoken
- Inference params: temperature, top-p, top-k, repetition penalty
- Prompt engineering: zero-shot, few-shot, chain-of-thought, ReAct
- OpenAI / Anthropic / Gemini API — rate limits, cost, context windows
- System prompts, message roles, and structured outputs (JSON mode)

■ OpenAI docs · Anthropic docs · 'Prompt Engineering Guide' (promptingguide.ai)

WHAT TO BUILD

- Build a multi-turn chatbot with OpenAI API from scratch (no wrappers)
- Experiment: how does temperature=0 vs 1.2 change outputs on same prompt?
- Project: CLI tool that summarises PDFs with structured JSON output

■ SKIP: Every Coursera GenAI certificate — they teach the same content at 10× the time

Phase 2 · RAG — Retrieval-Augmented Generation

Weeks 4–7

WHY THIS PHASE

RAG is the #1 skill asked in 'AI Engineer' JDs right now. It's also a prerequisite for building agents with persistent memory. Do this before agents.

WHAT TO LEARN

- Embeddings: what they are, cosine similarity, embedding models (text-embedding-3, BGE, E5)
- Vector DBs: Chroma (local dev), Pinecone or Weaviate (prod), FAISS basics
- Chunking strategies: fixed, recursive, semantic, late chunking
- Retrieval: dense, sparse (BM25), hybrid — when each wins
- Re-ranking: cross-encoders, Cohere Rerank
- Evaluation: context recall, faithfulness, answer relevancy (RAGAS framework)

■ LlamaIndex docs · LangChain docs · RAGAS GitHub · Pinecone learning center

WHAT TO BUILD

- Build a RAG pipeline over a 100-page PDF with LangChain or LlamaIndex
- Benchmark 3 chunking strategies on same docs — measure retrieval quality
- Project: 'Ask your codebase' tool — embed a GitHub repo, query it

■ SKIP: Building your own vector store from scratch — FAISS for learning, Chroma/Pinecone for everything else

Phase 3 · LLM Agents & Tool Use

Weeks 8–12

WHY THIS PHASE

Agents are where 80% of the job market money is now. RAG is the memory layer; agents are the action layer. You need both.

WHAT TO LEARN

- ReAct pattern: reason + act loops, stopping conditions
- Function / tool calling: OpenAI tools API, Anthropic tool use
- Agent frameworks: LangGraph (recommended), CrewAI, AutoGen — pick ONE
- Memory types: in-context, external (vector), episodic, semantic
- Multi-agent: orchestrator + worker pattern, handoffs
- MCP (Model Context Protocol) — emerging standard, worth knowing
- Common tools: web search, code interpreter, file I/O, APIs

■ LangGraph docs · LangChain agents guide · 'Building LLM Powered Applications' book

WHAT TO BUILD

- Build a ReAct agent with 3 tools: web search + calculator + code exec
- Implement a supervisor + 2 specialist agents with LangGraph
- Project: autonomous research agent that writes a structured report

■ SKIP: AutoGen for your first agent — LangGraph gives you more control and is more hireable

Phase 4 · Fine-tuning & Model Adaptation

Weeks 13–16

WHY THIS PHASE

Most people do this too early. Fine-tuning is for when prompting + RAG genuinely can't solve the problem. Know it for interviews; reach for it last in practice.

WHAT TO LEARN

- When to fine-tune vs. prompt engineer — the honest decision tree
- LoRA and QLoRA — parameter-efficient fine-tuning (the only ones that matter)
- Supervised fine-tuning (SFT): data format, training loop
- RLHF / DPO basics — preference tuning without a reward model
- Datasets: how to create high-quality instruction-following data
- Hugging Face ecosystem: transformers, PEFT, trl, datasets
- Serving fine-tuned models: vLLM, Ollama, Together.ai

■ Hugging Face PEFT docs · axolotl GitHub · Tim Dettmers LoRA paper · mlabonne/llm-course

WHAT TO BUILD

- Fine-tune Llama 3 8B on a custom dataset with QLoRA on Colab
- Compare: base vs. fine-tuned on your task — measure the actual delta
- Project: domain-specific assistant fine-tuned on real documents

■ SKIP: Full fine-tuning anything — QLoRA is what the industry uses; full FT needs A100s you don't have

Phase 5 · Production & Observability

Weeks 17–20

WHY THIS PHASE

The single biggest gap between self-taught and hired. Interviewers ask 'how do you know your RAG is working in prod?' Most candidates go blank. Don't be that person.

WHAT TO LEARN

- LLM observability: LangSmith, Langfuse, Arize Phoenix — tracing calls
- Evaluation pipelines: offline evals, online A/B, LLM-as-judge
- Guardrails: input/output validation, toxicity, PII detection (Guardrails AI, NeMo)
- Latency optimization: streaming, caching (semantic cache), batching
- Cost management: token counting, model routing (cheap model first)
- Deployment: FastAPI + async, Docker, basic Kubernetes awareness
- Vector DB ops: indexing pipelines, re-indexing strategies, embedding drift

■ LangSmith docs · Langfuse docs · 'LLM Engineer's Handbook' (Packt) · Eugene Yan's blog

WHAT TO BUILD

- Instrument your Phase 3 agent with LangSmith — trace every LLM call
- Build an eval harness: 50 Q&A; pairs, automated scoring with LLM judge
- Project: deploy a RAG API with streaming, auth, and cost logging

■ SKIP: MLflow for LLM tracing — it's built for classical ML; LangSmith/Langfuse are purpose-built

Phase 6 · Portfolio Project — The Interview Getter

Weeks 21–24

WHY THIS PHASE

One polished end-to-end project beats 10 tutorials on your resume. Recruiters take 90 seconds. Make it count.

WHAT TO LEARN

- Pick a real problem (not 'chat with PDF' — every candidate has that)
- Stack: FastAPI backend + LangGraph agent + vector DB + React/Streamlit UI
- Must-haves: eval suite, observability dashboard, cost metrics visible
- README with architecture diagram, benchmark results, deployment instructions
- Write a 500-word technical blog post about what broke and why

■ Your own brain + everything from Phases 1–5

WHAT TO BUILD

- Ideas that stand out: competitive intelligence agent, code review agent, automated due diligence for legal docs, customer support triage with escalation
- Deploy it — Railway, Render, or Fly.io. Free tier is fine.
- Record a 3-min Loom walkthrough — most interviewers won't run it locally

■ SKIP: Building another chatbot. Seriously.

THE HONEST JOB MARKET PICTURE

Role	Phases you need	What they actually test
AI Engineer (most roles)	1 · 2 · 3 + Phase 6 project	RAG pipeline, tool-calling agent, production thinking
ML Researcher	4 heavily + 1 · 2	Fine-tuning, RLHF, evaluation methodology, paper reading
LLM Platform Engineer	5 deeply + 1 · 2 · 3	Observability, cost optimisation, infra, latency
Applied Scientist	All 6, 4 & 5 especially	End-to-end build + eval + fine-tune + deploy

WHAT ACTUALLY WASTES YOUR TIME

■ **Certifications (DeepLearning.AI, Coursera GenAI)**

Fine for concepts. Won't get you hired alone. They teach what to think, not what to ship.

■ Framework wars (LangChain vs LlamaIndex vs Haystack)

Pick one orchestration layer. Ship with it. The framework doesn't appear on your offer letter.

■ Learning every transformer variant

Know the families. Understand why decoder-only dominates. Skip memorising BERT vs T5 vs Falcon internals.

■ Fine-tuning too early

Most 'fine-tuning needed' problems are actually prompt engineering + RAG problems. Prove that first.

■ Chasing every new paper/model drop

Follow 3–4 reliable sources. Build more than you read. The ratio should be 70/30 build/read.

TIMELINE AT A GLANCE

Phase 1	Phase 2	Phase 3	Phase 4	Phase 5	Phase 6
Wks 1–3	Wks 4–7	Wks 8–12	Wks 13–16	Wks 17–20	Wks 21–24
Foundations	RAG	Agents	Fine-tune	Production	Portfolio

■ Target: be interview-ready after Phase 3 + a strong Phase 2 project. Phases 4–5 push you to mid-level compensation. Phase 6 is what gets the interview call.